DineSync: A Restaurant & Bar Management System

Final Project Report
Bariscan Yolcu
SQL Database Design and Implementation Project

Abstract

DineSync is a robust and scalable database management system designed to streamline operations in a restaurant and bar environment. The system efficiently manages customer

profiles, staff roles, food and drink menus, orders, detailed order items, and payments through a well-organized relational structure. Our primary goal was to create a database model that not only enforces referential integrity and minimizes redundancy but also remains highly scalable and adaptable to future business needs. By maintaining a clean and normalized design, DineSync makes it easy to expand functionality, for example, by adding new modules such as reservations, loyalty programs, or employee scheduling without major restructuring. The system supports critical business operations and real-world workflows through structured SQL operations and optimized relationships. Challenges addressed during the project included designing logical polymorphic associations, creating realistic data models, and ensuring that the system could accommodate multiple user roles such as servers, bartenders, and managers, each interacting with the database in meaningful ways.

Mission Statement

The mission of DineSync is to design and implement a comprehensive, scalable, and future-ready database system that models the real-world operational needs of a restaurant and bar environment. The system enables efficient management of customers, staff roles, food and drink menus, orders, detailed order items, and payment transactions. Through a normalized relational structure, polymorphic item handling, and realistic business workflows, DineSync ensures data integrity, supports critical day-to-day operations, and allows seamless future expansion into areas such as reservations, loyalty programs, and inventory management, all while maintaining clarity, accuracy, and streamlined functionality.

Mission Objectives

- Design an entity-relationship (ER) model accurately reflecting the complex workflows of restaurant and bar operations, including customers, staff, food and drink menus, orders, order items, and payments.

- Normalize all relations to Third Normal Form (3NF) to eliminate redundancy, maintain data integrity, and support efficient query performance.

- Implement polymorphic associations within the model to handle different types of menu items under unified order item management.

- Populate the database with realistic and varied sample data modeled after real-world restaurant and bar scenarios.

- Implement complete CRUD (Create, Read, Update, Delete) operations and aggregate queries across all major entities and relationships using SQL.

- Validate system functionality through structured manual test plans and a comprehensive demo walkthrough.

- Demonstrate scalability and future extensibility of the database, including the potential for new modules such as reservations, loyalty programs, and employee scheduling.

E/R Diagram and Entity Overview

The DineSync database was modeled using an Entity-Relationship (E/R) diagram to clearly define the structure of the restaurant and bar management system. This diagram specifies all primary keys, foreign key relationships, and multiplicities between entities.

The primary entities and their relationships are outlined below:

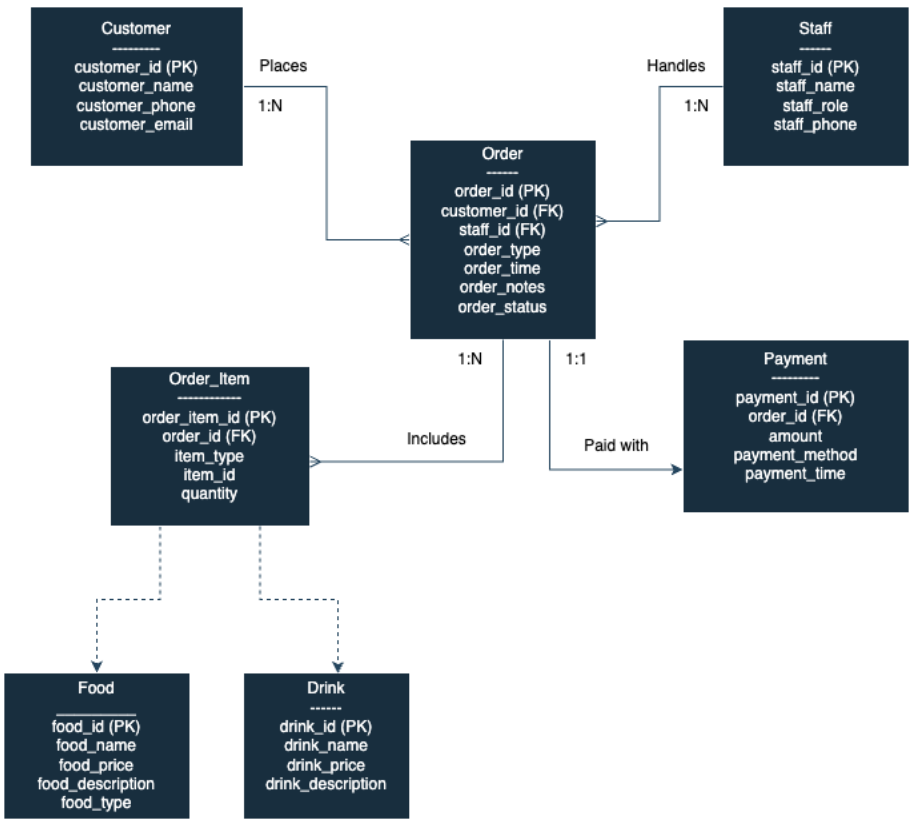
- **Customer**:
  - Primary Key: customer_id
  - Relationship: A Customer places one or more Orders (1:N).
- **Staff**:
  - Primary Key: staff_id
  - Relationship: A Staff member handles one or more Orders (1:N).
- **Order**:
  - Primary Key: order_id
  - Foreign Keys: customer_id (references Customer), staff_id (references Staff)
  - Relationships:
    - Each Order includes one or more Order Items (1:N).
    - Each Order is paid with exactly one Payment (1:1).
- **Order_Item**:
  - Primary Key: order_item_id
  - Foreign Key: order_id (references Order)
  - Special Logical Link:
    - Each Order Item refers to either a Food or Drink item.
    - This is a polymorphic association, handled through two attributes: item_type ('Food' or 'Drink') and item_id.
- **Food**:
  - Primary Key: food_id
  - Relationship:
    - Order Items that are food items reference Food entries via item_id when item_type = 'Food'.
- **Drink**:
  - Primary Key: drink_id
  - Relationship:
    - Order Items that are drink items reference Drink entries via item_id when item_type = 'Drink'.
- **Payment**:
  - Primary Key: payment_id
  - Foreign Key: order_id (references Order)
  - Relationship: Each Payment is associated with exactly one Order (1:1).

Relationship Multiplicities

- **Customer → Order**: One-to-Many (1:N)
- **Staff → Order**: One-to-Many (1:N)
- **Order → Order_Item**: One-to-Many (1:N)
- **Order → Payment**: One-to-One (1:1)
- **Order_Item → Food/Drink**: Optional Logical Links (Dashed lines in the diagram)

Note:

In the DineSync E/R diagram (shown below), primary keys are indicated with "(PK)" and foreign keys with "(FK)". Logical polymorphic links between Order_Item and menu items (Food or Drink) are indicated with dashed lines, as these are conditional based on item_type.



This E/R model establishes a strong foundation for enforcing referential integrity, supporting flexible menu structures, and enabling efficient query operations across all business activities within the restaurant and bar.

Normalization (1NF → 2NF → 3NF)

The DineSync database design carefully follows normalization principles to achieve Third Normal Form (3NF) across all tables. Each entity was analyzed to ensure atomicity, removal of partial dependencies, and elimination of transitive dependencies. This rigorous normalization process improves data integrity, reduces redundancy, and optimizes database performance for transactional queries.

Below is a detailed analysis of normalization steps applied to each major entity:

Customer Table

- **1NF:** Each attribute (customer_name, customer_phone, customer_email) holds atomic, indivisible values.
- **2NF:** Since customer_id is the primary key and uniquely identifies each record, all other fields are fully functionally dependent on customer_id.
- **3NF:** No transitive dependencies exist; customer contact information is directly related only to customer_id.

Staff Table

- **1NF:** All attributes are atomic — staff_name, staff_role, and staff_phone contain single, specific values.
- **2NF:** All non-primary attributes depend fully on staff_id, the table's primary key.
- **3NF:** No transitive dependencies exist. Staff role and phone information directly associate with each unique staff_id.

Food Table

- **1NF:** Food items are recorded with atomic fields (food_name, food_price, food_description, food_type).
- **2NF:** Every food attribute depends entirely on food_id as the primary key.
- **3NF:** No non-key attribute depends on another non-key attribute. Food name, type, and price are functionally dependent only on food_id.

Drink Table

- **1NF:** Drink items also maintain atomic attributes.
- **2NF:** Each drink's name, description, and price are dependent solely on drink_id.
- **3NF:** No transitive relationships exist; all fields are functionally dependent on drink_id alone.

Order Table

- **1NF:** Each order's type, time, notes, and status fields contain single values.

- **2NF:** customer_id and staff_id are foreign keys, linking to Customer and Staff tables respectively; all attributes relate fully to order_id.
- **3NF:** No transitive dependency appears — order_status, order_time, etc., are attributes directly related only to order_id.

Order_Item Table

- **1NF:** Order_Item maintains atomic entries for item_type and item_id, clearly specifying whether a food or drink was ordered.
- **2NF:** The non-key fields (item_type, item_id, quantity) depend fully on order_item_id.
- **3NF:** No attributes depend on another non-primary attribute; each field remains dependent only on order_item_id.

Payment Table

- **1NF:** Each payment record contains atomic entries for amount, method, and timestamp.
- **2NF:** All payment fields are fully dependent on payment_id, the primary key.
- **3NF:** No transitive dependencies exist, as payment_method and amount relate only to payment_id.

Relational Model Schema and Dependencies

Following the normalization process, the fully structured relational schema for the DineSync database is as follows:

- **Customer** (<u>customer_id,</u> customer_name, customer_phone, customer_email)
  - *Primary Key*: ***customer_id***
  - Each customer is uniquely identified by a customer ID. Customer contact details are atomic, satisfying normalization rules.

- **Staff** (<u>staff_id</u>, staff_name, staff_role, staff_phone)
  - *Primary Key*: ***staff_id***
  - Staff members are uniquely identified and assigned specific roles (Server, Bartender, Manager) through an ENUM field for easy role management.

- **Food** (<u>food_id</u>, food_name, food_price, food_description, food_type)
  - *Primary Key*: ***food_id***
  - Food items have associated prices, descriptions, and type categories (Appetizer, Main, Side, Dessert).

- **Drink** (<u>drink_id</u>, drink_name, drink_price, drink_description)
  - *Primary Key*: ***drink_id***
  - Drink items include various beverages with names, prices, and brief descriptions.

- **Order** (order_id, customer_id (FK), staff_id (FK), order_type, order_time, order_notes, order_status)
    - *Primary Key*: ***order_id***
    - *Foreign Keys*:
        - customer_id → Customer(***customer_id***)
        - staff_id → Staff(***staff_id***)
    - Orders are placed by customers and handled by staff, with fields capturing order type (Dine-In, Takeout), timestamps, notes, and status (In Progress, Completed).

- **Order_Item** (order_item_id, order_id (FK), item_type, item_id, quantity)
    - *Primary Key*: ***order_item_id***
    - *Foreign Key*:
        - order_id → Order(***order_id***)
    - This table handles the individual items associated with an order. The item_type field distinguishes between Food and Drink items, implementing a polymorphic association.

- **Payment** (payment_id, order_id (FK), amount, payment_method, payment_time)
    - *Primary Key*: ***payment_id***
    - *Foreign Key*:
        - order_id → Order(***order_id***)
    - Each payment is directly tied to a corresponding order, ensuring accurate tracking of financial transactions. Payment methods are categorized as Cash, Card, or Mobile.

Integrity Constraints

- Primary keys (underlined) uniquely identify each record within their respective tables.
- Foreign keys enforce relationships between related tables, ensuring referential integrity.
- Deletion or updates in parent tables (Customer, Staff, Order) are restricted appropriately to maintain data consistency.

This relational model ensures that the DineSync database is fully normalized, extensible, and reliable for future restaurant and bar operations.

List of Important Use Cases (with SQL statements the system will execute in the background):

Note:
The following use cases are designed under the assumption that a future user interface (UI) will be developed to interact with the DineSync database. While some steps in the use cases refer to actions such as "logging into the system" or "opening a form," these are conceptual and describe how users would eventually interact with the database through an application interface. For the current scope of this project, our focus is exclusively on the underlying database operations that would support such interactions. Each use case highlights the corresponding SQL commands that

would execute behind the scenes within the database management system, simulating the critical functionalities that a fully developed software solution would rely upon.

Use Case 1: Customer Registration by Receptionist

- Actor: Receptionist
- Steps:
    1. Receptionist logs into system.
    2. Opens "Add Customer" form.
    3. Inputs customer name, phone, and email.
    4. Submits form, system inserts into Customer table.

SQL Command for Use Case 1:

```
INSERT INTO Customer (customer_name, customer_phone, customer_email)
VALUES ('Chris Evans', '5559998888', 'cevans@example.com');
```

Use Case 2: Staff Role Update by Manager

- Actor: Manager
- Steps:
    1. Manager accesses "Staff Management."
    2. Selects staff member.
    3. Updates the staff_role.
    4. System updates Staff table accordingly.

SQL Command for Use Case 2:

```
UPDATE Staff
SET staff_role = 'Manager'
WHERE staff_id = 2;
```

Use Case 3: Creating a New Dine-In Order by Server

- Actor: Server
- Steps:
    1. Server logs into terminal.
    2. Opens "New Order."
    3. Assigns customer ID.
    4. Adds food and drink items.
    5. Submits order into Order and Order_Item tables.

SQL Command for Use Case 3:

```
INSERT INTO `Order` (customer_id, staff_id, order_type, order_time, order_notes,
order_status)
VALUES (4, 1, 'Dine-In', NOW(), 'Allergic to peanuts', 'In Progress');
```

Use Case 4: Deleting an Order Item by Server

- Actor: Server
- Steps:
    1. Server views active order.
    2. Selects unwanted item.
    3. Deletes it.
    4. System removes item from Order_Item table.

SQL Command for Use Case 4:

```
DELETE FROM Order_Item
WHERE order_item_id = 5;
```

Use Case 5: Aggregating Sales by Payment Method by Manager

- Actor: Manager
- Steps:
    1. Manager opens reporting dashboard.
    2. Runs a payment summary grouped by payment method.

SQL Command for Use Case 5:

```
SELECT payment_method, SUM(amount) AS total_revenue
FROM Payment
GROUP BY payment_method;
```

Use Case 6: Viewing Customer Order History by Manager

- Actor: Manager
- Steps:
    1. Manager opens Customer History section.
    2. Searches by customer name or ID.
    3. Views all associated orders.

SQL Command for Use Case 6:

```
SELECT Customer.customer_name, `Order`.order_id, `Order`.order_status, `Order`.order_time
FROM Customer
JOIN `Order` ON Customer.customer_id = `Order`.customer_id
WHERE Customer.customer_id = 4;
```

E/R Diagram Summary

The ER model features seven main entities: Customer, Staff, Food, Drink, Order, Order_Item, and Payment. Key relationships are:

- One-to-many (1:N) between Customer and Order.
- One-to-many (1:N) between Staff and Order.
- One-to-many (1:N) between Order and Order_Item.
- Logical association between Order_Item and either Food or Drink.
- One-to-one (1:1) between Order and Payment.

Polymorphic association between Order_Item and menu items is handled through item_type and item_id attributes.

Summary of SQL Operations Tested

Test ID	Entity/Table(s)	Operation Type	SQL Example	Expected Outcome
T1	Customer	Insert	`INSERT INTO Customer (customer_name, customer_phone, customer_email) VALUES (...);`	New customer record inserted successfully
T2	Staff	Update	`UPDATE Staff SET staff_role = 'Manager' WHERE staff_id = 2;`	Staff role updated correctly
T3	Food	Aggregate (Grouping)	`SELECT food_type, COUNT(*) FROM Food GROUP BY food_type;`	Food items counted by category
T4	Order_Item	Delete	`DELETE FROM Order_Item WHERE order_item_id = 5;`	Order item deleted without error
T5	Payment	Insert	`INSERT INTO Payment (order_id, amount, payment_method) VALUES (...);`	New payment recorded successfully
T6	Payment	Aggregate (Summation)	`SELECT payment_method, SUM(amount) FROM Payment GROUP BY payment_method;`	Total payments

Test ID	Entity/Table(s)	Operation Type	SQL Example	Expected Outcome
				summarized by method
T7	Customer + Order	Join Query	`SELECT Customer.customer_name, Order.order_id FROM Customer JOIN Order ON Customer.customer_id = Order.customer_id;`	Customer names matched with their orders
T8	Order + Payment	Join Query	`SELECT Order.order_id, Payment.amount FROM Order JOIN Payment ON Order.order_id = Payment.order_id;`	Orders correctly matched with payments
T9	Order + Order_Item	Join Query	`SELECT Order.order_id, Order_Item.item_type, Order_Item.item_id FROM Order JOIN Order_Item ON Order.order_id = Order_Item.order_id;`	Items correctly associated with their orders

Sample Data Strategy

To simulate realistic business operations, the sample data for DineSync was carefully designed using real-world references and common industry practices.
Menu items were inspired by offerings from major casual dining chains such as Chili's, Applebee's, and TGI Friday's, reflecting a balanced mix of appetizers, main courses, sides, desserts, and a range of beverages including soda, coffee, beer, wine, and cocktails.
Each food and drink item was crafted with a name, price, and description that mirrors real-world expectations, helping to create a convincing menu dataset.

Staff roles were deliberately assigned based on typical restaurant structures, ensuring coverage across essential operational areas: Servers, Bartenders, and Managers. Employees were given realistic names and contact information to enhance authenticity.

Customer profiles were created using plausible names, valid email address formats, and standardized North American phone number structures. Orders were generated to include a variety of Dine-In and Takeout scenarios, ensuring full testing coverage of different operational flows.

Payments were diversified across Cash, Card, and Mobile transaction types to represent modern payment behaviors, with amounts matching the pricing of associated orders.

By aligning the sample data closely with real-world dining experiences, we ensured that queries, transactions, and test scenarios authentically reflect the types of activities encountered in a restaurant and bar environment. This approach provided a strong foundation for system validation, query testing, and the final demo presentation.

Demo Walkthrough Video (MySQL)

1. Show creation of all tables and review code.
2. Insert manual data in all of the tables (10+ entries per table) and review code.
3. Display data in all tables and review code.
4. Demo AGGREGATE Functions:
    1. Total customers are registered in system from customers table and review code.
    2. Total staff members by role in staffs table and review code.
    3. Average price of food items in food table and review code.
    4. Highest price among all drinks in drinks table and review code.
    5. Number of orders by order type from order table and review code.
    6. Total quantity of items ordered in order_item table and review code.
    7. Total revenue collected from all payments from payments table and review code.
5. Demo JOIN Statements/Relationships:
    1. List of customer names with their order IDs and review code. (Customer and Order table join)
    2. List of staff names who handled each order and review code. (Staff and Order table join)
    3. List of order IDs with their ordered item types and quantities and review code. (Order and Order_Item table join)
    4. List of order IDs with payment amounts and payment methods and review code. (Order and Payment table join)
6. Bonus section: Demo potential expansion project idea
    1. Create new table for database called "Reservation" and review code.
    2. Example data inserted into Reservation table to run queries off of and review code.
    3. Example Query to View Reservations for specific customers (Join Reservation and Customer table) and review code.
    4. Display results and discuss how easy it is to expand on the database based on restaurant needs. Very scalable.

Link to demo video presented by Bariscan Yolcu:

https://drive.google.com/file/d/19OdRyGoEVfMNmq5bQgzQQzNVuYUACqqO/view?usp=sharing

Conclusion

The DineSync database project successfully models the key operational processes of a modern restaurant and bar using a fully normalized relational schema. Through careful entity design, referential integrity enforcement, and structured normalization, the system minimizes redundancy, maintains data accuracy, and supports a wide variety of business activities.

By building out realistic use cases and implementing SQL operations for inserting, updating, deleting, and querying data, DineSync demonstrates a robust framework capable of handling

real-world restaurant workflows. The system accommodates role-specific tasks for servers, bartenders, and managers, ensuring that all critical day-to-day operations are covered.

The normalized design also successfully promotes future scalability. DineSync can easily be extended to support additional modules such as reservation management, customer loyalty programs, inventory tracking, and employee scheduling. These potential enhancements would allow DineSync to evolve from a core operational database into a comprehensive, full-service restaurant management platform adaptable for a wide range of restaurant sizes and business models.

Overall, this project not only fulfilled all academic requirements but also established a strong foundation for a real-world application, capable of scaling alongside growing operational needs in the hospitality industry.

References

- Chili's Grill & Bar, Applebee's, and similar American casual dining restaurant menus. (2025). *Sample menu concepts and structures adapted for project use.*

- University of Houston-Downtown Course Materials. (2025). *Database normalization guidelines and best practices.* Database Management Systems, CS5318.

- MySQL Documentation. (2025). *MySQL 8.0 Reference Manual.* Retrieved from: https://dev.mysql.com/doc/

- Draw.io (diagrams.net). (2025). *Free Online Diagram Software for E/R Diagram Creation.* Retrieved from: https://www.diagrams.net/